

Chapter 2: Layout

As described in Chapter 1, an RET solution is mainly a **computer algorithm** which converts a **design layout** to a **mask layout**. An RET engineer must know layout inside out.

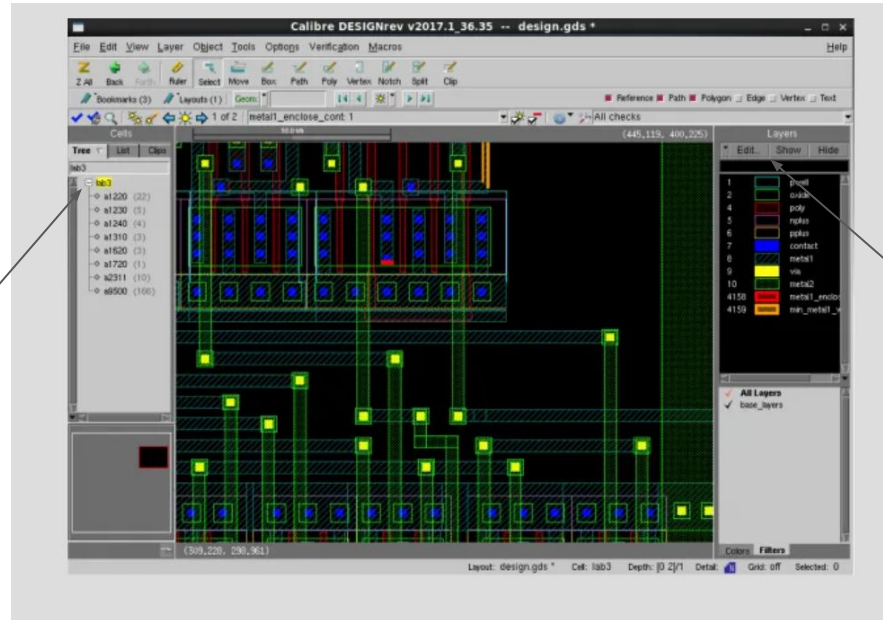
Digital Representation of a Layout: A binary layout file, i.e., myLayout.gds, myLayout.oas(, etc., can be handled like ordinary file in UNIX/Linux environment, for example, zipped, tarred, ftp,?) etc.

Graphic Representation of a Layout: A layout can be viewed/edited graphically on an EDA platform, for example, Calibre as shown in next slide. We shall discuss much details of layout graphics later.

Text Representation of a Layout: Although a layout is most useful and often saved in a binary gds/oasis format, it can also be represented in a text file format, in which layout hierarchy, geometry, and all other informations are humanly readable.

Layout Structure: Hierarchy and Layer

A layout file size can be as small as a few bytes or as big as tens of gigabytes. Regardless of the file size, data in any layout is organized through hierarchy and layer.



Hierarchy of cells. A cell is a collection of shapes.

List of layers. Any shape is drawn upon just one layer.

Layout Layer

Layer: A layer is a collection of shapes. These shapes can be displayed on a graphic viewer.

Layers are defined as “Layer Name (layer number, datatype number).” Here Layer Name is a string, layer number and datatype number are positive integers, from 0 to 256. For example, metel1 (31, 0).

Layers in Design Layout and Mask Layout:

In a design layout, each layer corresponds to a **device layer**, i.e., layer 31 is for metal 1. Often times, more than one datatype is needed. A datatype is a label to indicate the usage of a shape, for example, metal (31,0) and metal (31, 1) are shapes to be processed by OPC and Not by OPC, respectively; metal (31, 32) is for shapes in memory cells.

A mask layout has just one layer. This layer includes all shapes that will be written (by mask shop) on the mask, including shapes of devices, test patterns, metrology patterns, mask labels; shapes that have gone thru OPC/ILT or not; shapes that are **target or assisting or dummy**, which will be explained later.

Layout Hierarchy

Cell: Cell is the building block of layout hierarchy. Any shape is associated with just one cell. A cell name is a unique and case-sensitive string.

Any layout contains one or multiple **top** cell(s). A top cell contains **references of sub-cells one level below**, as well as **shapes in any number of layers**. The references are location/orientation of these sub cells, and the shapes are associated with the top cell.

Each sub-cell contains references to its sub-cells one level below as well as shapes, and so on so forth.

A **leaf** cell is at the lowest level of hierarchy; it contains only shapes and no reference.

A cell is self-containing, meaning it can be streamed out as an individual layout file.

Depth of Hierarchy: Depth is the number of levels between the top cell to the cell of interest. For example, “depth = 0” refers to top cell; “depth =1” refers to one level down.

As we shall see later, when editing shapes, user should specify one or a range of hierarchy level(s). Only shapes associated with cells at those level(s) can be edited. For example, “select level = 1, or 1-5.”

Illustration of Layout Hierarchy and Layer

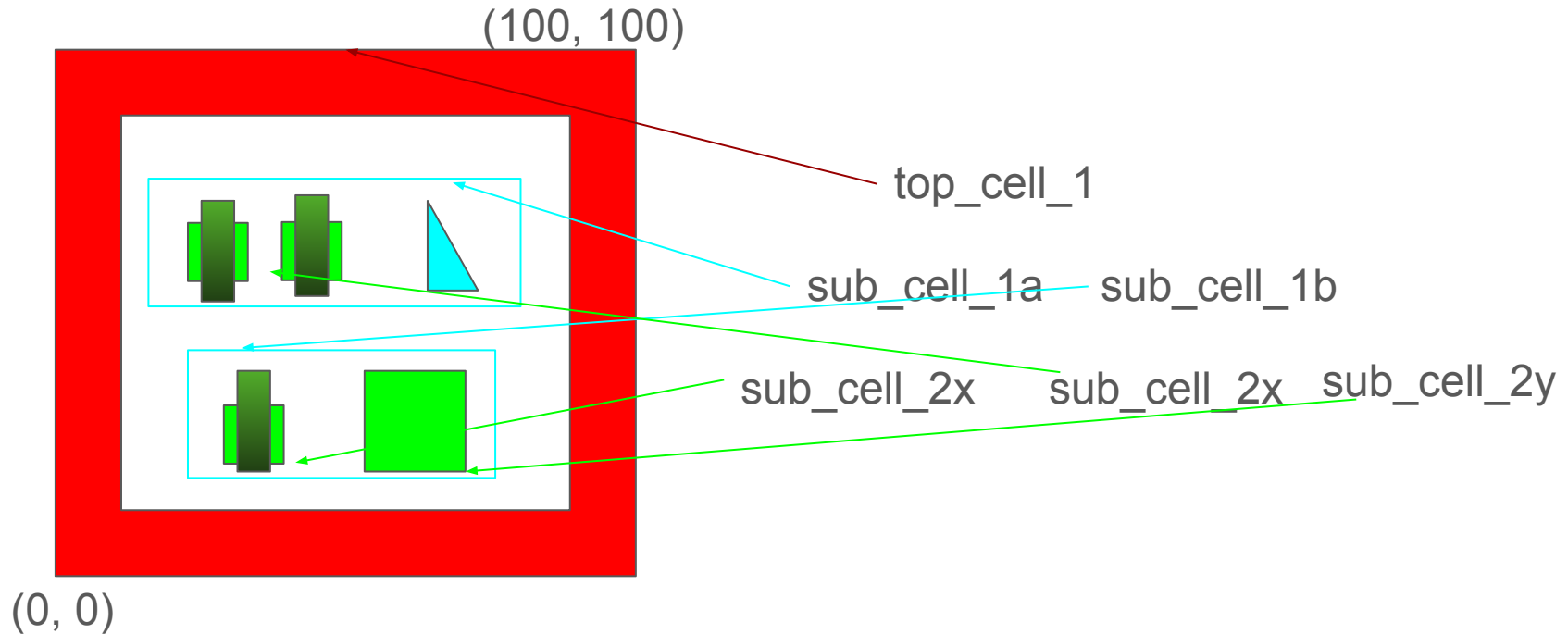


Illustration of Layout Hierarchy and Layer (continued)

Hierarchy Level	Cells	Cell Reference (x, y)	Shapes
0 (top cell)	Top_cell_1	sub_cell_1a: (0,25) sub_cell_1b: (0,-25)	Red drawn on layer/datatype (0,0)
1	sub_cell_1a	sub_cell_2x: (0, 0) sub_cell_2x: (-25, 0)	Blue drawn on layer/datatype (10,0)
1	sub_cell_1b	sub_cell_2x: (-20, 0) sub_cell_2y: (20, 0)	
2 (leaf cell)	sub_cell_2x	N/A	Solid green drawn on layer/datatype (15,0) Shaded green drawn on layer/datatype (20,0)
2 (leaf cell)	sub_cell_2y	N/A	Solid green drawn on layer/datatype (20,0)

Layout Precision

As in any digital graphics, shapes on a layout are drawn on **grid**. Grid size must be compatible with **precisions** required for the technology node and the layer.

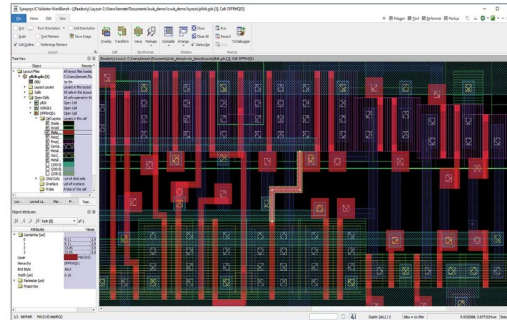
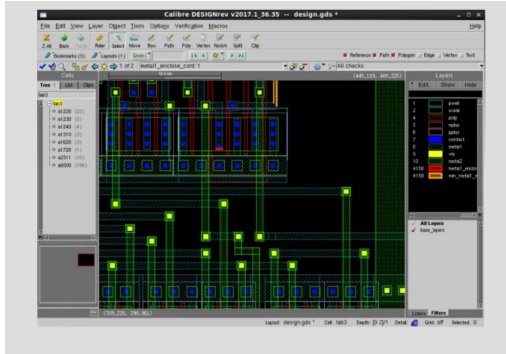
Grid: Physical location of a vertice of a polygon shape is $(X\ Y)*grid$, where $(X\ Y)$ are integrals. Each layer should define its own grid. For example, in a 7nm layout, a critical layer, such as poly, contact, should use very small grid, like 0.25nm, while a non-critical layer such as top metal can use 5nm grid.

Magnification: Often times, a layout layer created in physical design has a grid too big for the precision that RET aims to achieve. This design layer can be **magnified** by an integer **Mag**. On such layout, a physical location remains unchanged: $(X*Mag\ Y*mag)*grid/Mag$, while RET can edit vertices/edges with a precision grid/Mag. RET would output a mask layout in this reduced grid, that would be used in mask writing.

Limitation: In theory, mask layout grid can be reduced indefinitely by magnification. However, such grid reduction becomes meaningless beyond resolution of mask writers. In addition to technological limitations, time and cost of mask writing must also be considered when choosing a mask grid. Details will be addressed in Chapter 6 Mask Making. Overall, for mature technology node, mask grid should not go below 1nm. For advanced nodes, mask grid can be as low as 0.1nm.

Graphical Representation of Layout

As we have learned, a layout file is a blueprint in digital format. Loading the file onto a viewer GUI, the blueprint becomes graphics. Such viewer, layout workbench, is one of the most important tools provided by an EDA company. Here are snapshots of layout workbench by Siemens EDA (Mentor Graphics) and Synopsys.



Basics Functions/Features of Layout Workbench

Any layout workbench should provide these basic functions/features:

File Control: Browse/read/write of multiple layouts.

Window Control: Show/hide/move/size/arrange windows of layouts, simulation results (described later), toolbars, manual bars, and other contents.

Layout Control: Apply operations to a whole layout, including overlay, scale, rotate, mirror, XOR, etc.

Layer Control: Apply operations to a whole layer, including create/copy/delete/group/rename/mark/select layer(s), set layer attributes, i.e., color, pattern, /show/hide/set active layer(s), etc.

Cell Control: Display cell hierarchy; Select working cell; Set editability; Streaming out a cell; etc.

Basics Functions/Features of Layout Workbench (continued)

Shape Control: Viewing/editing individual shape are most useful, most demanding, and most nuanced operations.

View Functions: zoom, goto, pane, ruler, flagger, highlight, gauges

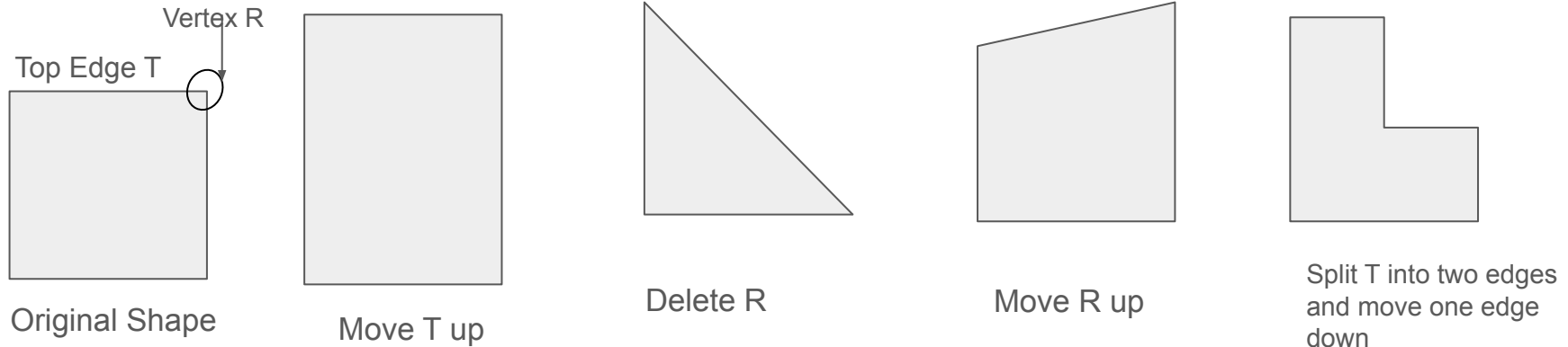
Edit Functions: select, drawn, copy/cut/paste/generate, move, split, etc; editing can be done to shape/edge/vertice.

Options: Allow setting/saving default for parameters of all functions/features above. Examples: background color, text fonts, window size.

Help: Provide license and version information, user manual, release note, etc.

Examples of Layout Editing on Workbench

On a workbench, an RET engineer routinely reviews a whole layout or its portions. But manual editing of layout shapes is limited to simple, local, and occasional work such as fixing hotspots and creating test cases. As mentioned before, an RET solution converts design layout to mask layout with computer algorithms, not manual editing. Below are some common manual editing work.



Operations on Layout Workbench

Users can execute functions/features in multiple ways:

Use Icon/Mouse: Browse and move the mouse; Left/right/middle click on graphic icons w/wo combining with “shift” “ctrl” and function keys.

Use Hotkey: It is common and convenient to program hot keys to perform certain functions. For example, ctrl-”g” for drawing a gauge.

Use Macro: Some workbench allows user to run macros that can be built-in, imported, or composed inline. Such script is usually a combination of platform-specific commands and general programming languages, i.e., tcl, python, etc.

Industrial-Grade Layout Workbench

Modern, industrial-grade layout workbench should demonstrate following capabilities in order to stay competitive:

- Read in/write out/extract portions of layouts, with file size up to tens of gigabytes, in reasonable time; Support 32/64 bits and compressed/zipped formats, Distributed Processing; Caching; Allow monitoring of progress.
- Associate graphics with text files of gauges, highlights, hotspot locations, and metrology data for model calibration (to be described later).
- Allow personalization of the appearance, functionality, and ease-of-use.
- Keep up with advances in RET, for example, support all-angle polygon and multigon (ILT, Curve OPC); support extremely high precision (EUV).
- Must be agnostic to operating systems that the vendor promises to support. i.e., Red Hat, etc, and hardware environment, i.e., Intel/AMD CPU/GPU.
- Allow logging of work, even when a GUI session crashes. The log files are crucial for debugging and reproducing the work.
- Provide scripting capabilities that are GUI-equivalent in order to run scripts in batch jobs. i.e., layout loading, zoom, highlight creations, etc.
- Should be backward compatible in version control.
- Should pass rigorous QA.

Layout Operations through RET

As mentioned before, an RET algorithm is built upon a specific EDA platform. Although EDA companies provide different software libraries, they must include at least these basic layout operations:

Boolean operations; Flattening of cell; Topological characterization of polygon and edge, i.e. hole/donut, interaction via inside/outside/touch/overlap/see-thru, etc; Geometrical characterization, i.e., for polygon—density, area, perimeter, vertex count, etc; for edge—length, orientation, space, convex/concave.

As we shall see later, these operations provide basis for layout corrections.

Chapter 2: Summary

In this chapter we have learned the following basic concepts of layout: File format, Cell, Hierarchy, Layer, Precision. We have discussed in details how to process layout graphics on workbench, i.e., read/write/extract, view, edit.

RET is fundamentally manipulation of shapes of design layout, although most advanced RET has elements of “free-form” shape creation. Manipulation or creation of shapes are directed by **rules** or **simulations with models**. We shall discuss model and simulations in next chapter.